

B.5 Lab 5: LU Factorization

Problem 1: LU Factorization with Partial Pivoting

When using Gaussian Elimination to solve a linear system, partial pivoting is a crucial component that guards against division by zero when computing the factor. For this problem, we will reconstruct an actual CT image. Download `CTdata40x40.mat`. It contains the full coefficient matrix A and the right-hand side vector $\mathbf{b}$ needed to reconstruct a 40x40 CT image. Also, take a look at `gaussNoPivot.m`. This function contains a fully working version of Gaussian Elimination with no pivoting, exactly the code developed in Listing 4.3 of Chapter 4. Unfortunately, if you try to use `gaussNoPivot` to solve for the values in our 40x40 image, it does not go well:

```
>> load CTdata40x40.mat
>> x = gaussNoPivot(A,b);
>> disp(x)
NaN
NaN
NaN
NaN
NaN
NaN
NaN
NaN
```

NaN stands for “not a number” and it occurs in MATLAB when a number cannot be defined or is unrepresentable, such as when a division by zero takes place. Write a function `myLUfactorization.m` that takes as input a matrix A and returns its LU factors and the permutation matrix P satisfying $PA = LU$, as in Equation (4.14):

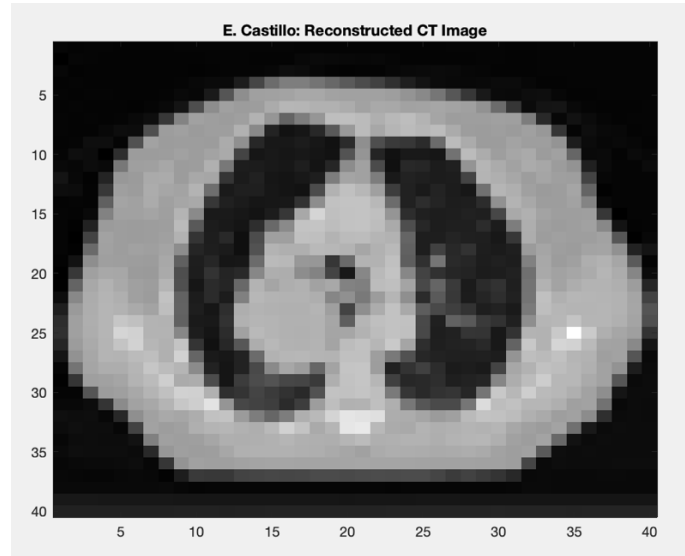
```
function [L,U,P] = myLUfactorization(A)
```

Recall, the U factor is the upper triangular matrix that results from forward elimination, while the lower triangular factor contains ones on the diagonal and the factors used during forward elimination. The permutation matrix encodes the partial pivoting row swaps. **Hint:** `gaussNoPivot.m` already implements the forward elimination loops. I suggest adjusting its code to perform the factorization, and be mindful of how pivoting might need to be accounted for when calculating L .

Once your factorization code is complete, use it to reconstruct the CT image. Since $PA\mathbf{x} = LU\mathbf{x} = P\mathbf{b}$, the solve amounts to two triangular systems, and for this problem you may solve them with MATLAB’s backslash operator. Run the following commands to visualize the reconstructed lung CT image:

```
>> load CTdata40x40.mat
>> [L,U,P] = myLUfactorization(A);
>> x = U \ ( L \ (P*b) );
>> img = zeros(40,40); img(:) = x;
>> close all; imagesc(img); colormap('gray');
>> title('E. Castillo: Reconstructed CT Image');
```

The image will look like below. Name it `CT40x40.pdf`.



Problem 2: Dynamic Imaging and LU Factorization

Many applications require solving a series of linear systems with the same coefficient matrix but different right-hand side vectors. In such instances, applying Gaussian Elimination repeatedly for each right-hand side vector is computationally inefficient (especially for large systems). It is more efficient to factorize the coefficient matrix first and then use the factorization to solve the system for each right-hand side vector. Recall the LU factorization of a matrix $A \in \mathbb{R}^{n \times n}$:

$$A = LU$$

where $L \in \mathbb{R}^{n \times n}$ is a lower triangular matrix and $U \in \mathbb{R}^{n \times n}$ is upper triangular. Given the LU factorization, a linear system can be solved with one forward substitution and one backward substitution:

$$A\mathbf{x} = \mathbf{b}$$

$$LU\mathbf{x} = \mathbf{b}$$

First, use forward substitution to solve:

$$L\mathbf{y} = \mathbf{b}$$

then, use backward substitution to solve for $\mathbf{x}$:

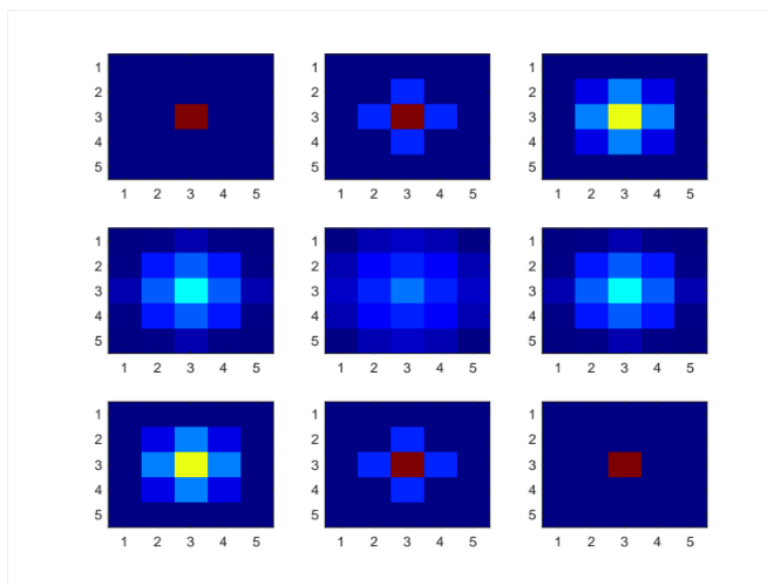
$$U\mathbf{x} = \mathbf{y}.$$

In the Dynamic Medical Image and Computing Lab,¹² we analyze organ motion within a temporally resolved image series (dynamic imaging). In this exercise, we will use our own LU factorization code, along with our own triangular solver code, to reconstruct a movie of an object moving. Specifically, we will reconstruct nine 2-D images that together represent the motion of an object as a function of time.

Problem 2: Take a look at `movieRecon.m`. It calls `CTmat5x5.p` and `CTrhs_time.p` to get the data needed to reconstruct a 5x5 image at specified time points. `movieRecon.m` is nearly complete. Three pieces are missing: the LU factorization and the two triangular solver subfunctions, `backSub` and `forwardSub`. For the factorization, call your `myLUfactorization` from Problem 1. `forwardSub`

¹²<https://sites.utexas.edu/dmiclab/>

takes in a lower triangular matrix, L , a right-hand side vector, $\mathbf{b}$, and uses forward substitution to solve for $\mathbf{y}$. `backSub` takes in an upper triangular matrix and solves for $\mathbf{x}$. Back substitution is already implemented in Listing 4.2 of Chapter 4, so include it as the `backSub` subfunction. Your task is to write the `forwardSub` subfunction yourself, without the backslash operator. The completed `movieRecon.m` will generate a figure showing all the movie frames:



General Submission Guidelines

<code>[FirstName][LastName]_[filename].m</code>	For programs
<code>[FirstName][LastName]_[filename]_plot_1.pdf</code>	For corresponding plots (export them to pdf)
<code>[FirstName][LastName]_[filename]_plot_2.pdf</code>	For problem with more than one plot

Submission Checklist

- Problem 1: `myLUfactorization.m`
 - Fully commented code and header
 - Correct pivot identification and row swaps recorded in P
 - Correct L and U factors
 - Correct reconstructed CT image with title
- Problem 2: `movieRecon.m`
 - Complete header
 - Correct implementation of forward substitution
 - Correct movie frames figure using your `myLUfactorization`